

08_mirror_ceiling

July 1, 2026

1 08 — Copy the exits too? The “perfect mirror” ceiling

Your point: notebook 07 only copied *entries* and held to resolution — it threw away the sharps’ **exit timing**, which is a big part of their profit. So can we copy exits as well?

Yes — and here we measure the ceiling of doing so. The `realizedPnl / totalBought` we already pull is each position’s *actual realized return, with the sharp’s own entries AND exits included*. Averaging that over point-in-time-qualified positions tells us what a copier would earn by mirroring them **perfectly** (same entries, same exits), equal-weighted and frictionless. It’s the best case — real copying is lagged and can only do worse.

We compare three things, per backer count: - **buy_hold_mean** — notebook 07’s method (enter, hold to resolution). The floor. - **mirror_mean / mirror_median** — mirror their entries *and* exits, equal stake. ← the answer to your question - **dollar_weight** — their actual aggregate return (mirror + *their sizing*). The full picture.

The gaps between these three localise exactly where their money comes from: resolution outcome vs exit timing vs position sizing.

```
[1]: import importlib, pmc
importlib.reload(pmc)
from pmc import CFG, get_leaderboard, get_closed_positions
import pandas as pd, numpy as np
print("gate:", CFG.WF_MIN_TRAILING_TRADES, "trades,", CFG.
      ↪WF_MIN_TRAILING_WINRATE, "win-rate")
```

gate: 30 trades, 0.55 win-rate

1.1 1. Pull history WITH realized PnL and cost basis

Same candidate pool as nb 06/07 (cached). We now also keep `realizedPnl` and `totalBought` — the two fields that encode their true, exit-inclusive return.

```
[2]: cands = {}
for w in ("ALL", "MONTH"):
    for r in get_leaderboard(window=w, limit=CFG.WF_CANDIDATES):
        cands.setdefault(r["wallet"], r)

rows = []
for i, wallet in enumerate(cands):
```

```

for p in get_closed_positions(wallet, max_positions=600):
    cost = float(p.get("totalBought") or 0)
    if cost <= 0:
        continue
    rows.append({
        "wallet": wallet, "conditionId": p.get("conditionId"), "outcome": p.
        ↳get("outcome"),
        "entry": float(p.get("avgPrice") or 0), "entry_ts": int(p.
        ↳get("timestamp") or 0),
        "endDate": p.get("endDate"),
        "realizedPnl": float(p.get("realizedPnl") or 0), "cost": cost,
        "won": 1 if float(p.get("realizedPnl") or 0) > 0 else 0,
    })
    if (i + 1) % 50 == 0:
        print(f"  pulled {i+1}/{len(cands)}")

h = pd.DataFrame(rows)
h["res_ts"] = (pd.to_datetime(h["endDate"], errors="coerce", utc=True).
    ↳astype("int64") // 10**9)
h = h[(h["entry_ts"] > 0) & (h["res_ts"] > 0) & (h["res_ts"] > h["entry_ts"]) &
    ↳(h["entry"] > 0)].copy()
h["their_roi"] = h["realizedPnl"] / h["cost"]          # actual return, exits_
    ↳included
print(f"{len(h)} usable positions | {h['wallet'].nunique()} wallets")

```

```

pulled 50/288
pulled 100/288
pulled 150/288
pulled 200/288
pulled 250/288
24255 usable positions | 232 wallets

```

1.2 2. Point-in-time qualification (same rule as nb 07)

```

[3]: h = h.sort_values("entry_ts").reset_index(drop=True)
qmask = np.zeros(len(h), dtype=bool)
for wallet, idx in h.groupby("wallet").groups.items():
    w = h.loc[idx]
    rs = w.sort_values("res_ts")
    rt = rs["res_ts"].values
    wc = np.cumsum(rs["won"].values)
    k = np.searchsorted(rt, w["entry_ts"].values, side="left")
    wins = np.where(k > 0, wc[np.clip(k - 1, 0, len(wc) - 1)], 0)
    wr = np.where(k > 0, wins / np.maximum(k, 1), 0.0)
    qmask[np.array(idx)] = (k >= CFG.WF_MIN_TRAILING_TRADES) & (wr >= CFG.
    ↳WF_MIN_TRAILING_WINRATE)
q = h[qmask].copy()

```

```
# backers per market-side among qualified entries
q["backers"] = q.groupby(["conditionId", "outcome"])["wallet"].
↳transform("nunique")
print(f"{len(q)} PIT-qualified positions across {q['conditionId'].nunique()}_
↳markets")
```

5748 PIT-qualified positions across 3430 markets

1.3 3. The decomposition — where does their money actually come from?

buy_hold uses their entry price and pays out only at resolution (nb 07 logic, no slippage here so it's a clean comparison). mirror is their real per-position return incl. exits.

```
[4]: def decompose(df, n):
    s = df[df["backers"] >= n]
    if s.empty:
        return None
    buy_hold = np.where(s["won"] == 1, (1 - s["entry"]) / s["entry"].
↳clip(lower=0.01), -1.0)
    return {
        "min_backers": n, "n_pos": len(s),
        "buy_hold_mean": round(np.mean(buy_hold), 3),
        "mirror_mean": round(s["their_roi"].mean(), 3),
        "mirror_median": round(s["their_roi"].median(), 3),
        "mirror_%pos": round((s["their_roi"] > 0).mean(), 3),
        "dollar_weight": round(s["realizedPnl"].sum() / s["cost"].sum(), 3),
    }

res = pd.DataFrame([r for r in (decompose(q, n) for n in [1, 2, 3]) if r])
print("PERFECT-MIRROR CEILING (frictionless, point-in-time qualified):")
res
```

PERFECT-MIRROR CEILING (frictionless, point-in-time qualified):

```
[4]:   min_backers  n_pos  buy_hold_mean  mirror_mean  mirror_median  mirror_%pos  \
0           1   5748         3.710         0.110         0.057         0.682
1           2  2382         3.529         0.182         0.163         0.750
2           3  1147         2.163         0.208         0.199         0.786

    dollar_weight
0           0.033
1           0.036
2           0.031
```

1.4 4. How to read it — and the decision

- **mirror_mean** and **mirror_median** clearly $> 0 \rightarrow$ copying exits recovers real edge that buy-and-hold missed. Your instinct was right, and the next step is a **realistic lagged mirror**

(notebook 09): enter shortly after their buy, exit shortly after their sell, at the prices actually available then, with slippage — to see how much survives the delay.

- **mirror_mean/median 0 (or only dollar_weight is positive)** → their profit is really from *sizing* (a few big bets) and survivorship, not from exit timing a copier can follow. Then mirroring exits won't save it and notebook 07's verdict stands.

1.4.1 Honest limits of this ceiling

- **Frictionless & instant.** It assumes you match their exact entry and exit prices. Real copying is lagged twice (entry *and* exit are public only after the fact), so the live number is lower — often much lower, since you'd be selling into the same exits.
- **Equal-weight their sizing.** `mirror_*` gives each position one vote; `dollar_weight` shows what their sizing added on top. You can copy *what* and *when*, but not their conviction-weighting, without knowing their bankroll live.
- **Survivorship** in the candidate pool still nudges everything up a little.

So `mirror_*` is the **optimistic ceiling** for copy-trading with exits. If even the ceiling is thin, the realistic strategy is thinner. If the ceiling is strong, notebook 09 tells us whether it survives contact with latency.